Advanced PHP

Hongxia (Helen) Wang
wangh25@aston.ac.uk

Introduction

Learning outcomes

- Working with databases
- Cookies & Sessions
- Authentication using sessions
- Know Object-Oriented PHP basic

Resources

- PDO: https://www.php.net/manual/en/book.pdo.php
- Password hashing: https://www.php.net/manual/en/book.password.php
- PHP sessions: https://www.w3schools.com/php/php_sessions.asp
- PHP cookies: https://www.w3schools.com/php/php_cookies.asp
- Chapter 14, Web programming Step by Step

Acknowledgement: some texts or slides are from the Web Programming Step by Step slides provided by Marty Stepp, J. Miller, and V. Kirst

Outline

- Working with databases
- Cookies & Sessions
- Authentication using sessions
- OO PHP
  - Procedural *vs* Object-Oriented
  - Define Classes and Objects

PHP: Connect to Databases

- Abstraction Layers
  - ODBC: Open DataBase Connection (Unified)
  - **PDO: PHP Data Objects**
- Vendor Specific Database Extensions
  - **MySQLi: MySQL**
  - OCI8: Oracle
  - MongoDB
  - PostgreSQL
  - SQLite

PDO: **PHP Data Object**

PDO: PHP Data Object. One class pre-defined by PHP. It is a database library allows you to connect to many different database programs

PDO Methods:

Name	Description
__construct	Creates a PDO instance representing a connection to a database.
query	performs a SQL SELECT query on the database returning a result set as a PDOStatement object
exec	performs a SQL query that modifies the database; return the number of affected rows
setAttribute	set various DB connection properties
quote	encodes a value for use within a query
prepare	prepares a statement for execution and returns a PDOstatement object

Querying a Database in PHP with PDO

- Call the constructor to create a **PDO Object**

```
new PDO($dsn, $username, $password);
```

  - **DSN** (Data Source Name) contains the information required to connect to the database. In a format:

```
"dbprogram:dbname=database;host=server"
```

- Example:

#connect to MySQL database **club** on localhost by default username/password

```
$db=new PDO("mysql:dbname=club;host=localhost", "root","");
```

query the **members** table

```
$rows=$db->query("SELECT * FROM members");
```

  - The **$rows** returned by PDO's **query** method is technically not an array but an object of type PDOStatement
  - The **members** table in the **club** database:

members (ID, username, password, programme, reason, gender)

PDOStatement object and methods

`PDOStatement`: Represents a prepared statement (SQL template) and, after the statement is executed, an associated result set.

Here are some of its methods:

`columnCount()`	number of columns in the results
`fetch()`	return the next row from a result set
`fetchColumn(number)`	return one column from the next row of result set
`rowCount()`	number of rows returned by the query
`bindParam($para,$var)`	Binds a parameter to the specified variable name
`execute()`	Executes a prepared statement

```
if ($rows->rowCount() > 0) {
  $first_row = $rows->fetch();
  ...
}
```

A PDO query example

The query method returns the PDOstatement object with all rows:

- each row is an associative array of [column name => value]
- example: $row["username"] gives the value of the username column

```
<?php
……
 $db = new PDO("mysql:dbname=club;host=localhost",“root",“");
 $rows = $db->query("SELECT * FROM members");
 foreach ($rows as $row) {
?>
  <li> Name: <?= $row[“username"] ?>,
       Programme: <?= $row[“programme"] ?> </li>
<?php
}
……
?>:
```

- Name: Helen, Programme: computer science
- Name: Ali, Programme: cyber security

SQL injection

- **SQL Injection**: A flaw where the user is able to inject arbitrary commands into your SQL query

```
$qstr = "SELECT username, programme FROM members
    WHERE username = '$name' AND password = '$password'";
```

- What if the user enters the password: `' OR '1'='1`

```
$qstr = "SELECT username, password, programme FROM members
   WHERE username = '$username'
   AND password = '' OR '1'='1'";
```

- What will the above query return? Why is this bad?

SQL injection

- Injected SQL can:
  - change the query to output others' data (revealing private information)
  - insert a query to modify existing data (increase bank account balance)
  - delete existing data (…; DROP TABLE students; …)
  - bloat the query to slow down the server (JOIN a JOIN b JOIN c ...)
  - ...
- You should not directly include variables or query parameters in a query
- They might contain illegal characters or SQL syntax to mess up the query

Quoting Variables

```
# get query parameter for username
$username = $db->quote($_POST["username"]);
$password = $db->quote($_POST["password"]);
$query = "SELECT username, gender, programme FROM members WHERE
        username=".$username. "AND password=".$password;
```

- Similar to securing against HTML injection, **escape the string before you include it in your query**
- Call PDO's quote method on any variable to be inserted
- quote method escapes any illegal chars and surrounds the value with ' quotes
- Prevents bugs and security problems in queries containing user input

Prepared Statements and Bound Parameters

A better secure approach:

PDO::prepare() — prepare SQL statements with bound parameters

PDOStatement::bindParam — Binds a parameter to the specified variable name

PDOStatement::execute — Executes a prepared statement

Three main advantages:

- **Reduces parsing time**: the preparation on the query is done only once (although the statement is executed multiple times)
- **Bound parameters minimize bandwidth to the server**: you need send only the parameters each time and not the whole query
- **immune to SQL injection**: because parameter values, which are transmitted later using a different protocol, need not be correctly escaped. If the original statement template is not derived from external input, SQL injection cannot occur.

Examples

The SQL statement can contain zero or more **named (:name) or question mark (?) parameter markers** for which real values will be substituted when the statement is executed.

Example1 using **named** parameter marker:

```
$sth=$db->prepare("INSERT INTO members(username, password, programme)
            VALUES(:username, :password, :programme) ");
$sth->bindParam(':username', $username, PDO::PARAM_STR, 10);
$sth->bindParam(':password', $password, PDO::PARAM_STR, 256);
$sth->bindParam(':programme', $program, PDO::PARAM_STR, 50);

$sth->execute();
```

Example2 using **?** parameter marker:

```
$sth=$db->prepare('SELECT username, programme FROM members WHERE
     gender = ?');
$sth->execute(array("female"));
```

Catching an exception

```
try {
   statement(s);
} catch (ExceptionType $name) {
   code to handle the error;
}
```

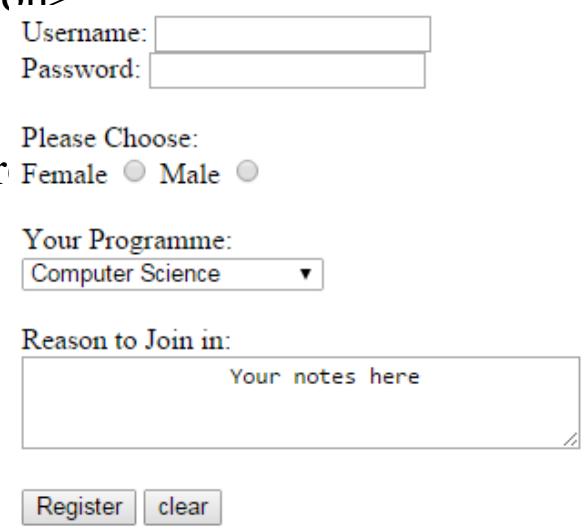
- a try/catch statement attempts to run some code, but if it throws a given kind of exception, the program jumps to the catch block and runs that code to handle the error

Examples with error handling

```
<?php
try{

  $db = new PDO("mysql:dbname=club;host=localhost",“root",“”);

  $rows = $db->query("SEELECT * FROM members ");

  foreach ($rows as row) { ... }

} catch (PDOException $ex) {
  //this catches the exception when it is thrown
  ?>

  <p>Sorry, a database error occurred. Please try again.</p>
  <p>(Error details: <?= $ex->getMessage() ?>)</p>

  <?php
}
…
```

Exercise: Register form

```
<form method = "post" action="register.php">
   Username: <input type="text" name="username" /><br /><br />
   Password: <input type="password" name="password" /><br /><br />
   Please Choose: <br>
   Female <input type="radio" value="female" name="gender" />
   Male <input type="radio" value="male" name="gender" /><br /><br />
   Your Programme: <br>
   <select name="programme">
       <option value="Computer Science" selected="selected"> Computer Science </option>
       <option value="Cyber Security"> Cyber Security </option>
       <option value="Joint programme "> Joint programme </option>
   </select><br/> <br/>
   Reason to Join in:
   <textarea name="Reason" rows="3" cols="40"> Your notes her
   </textarea> <br /><br />
   <input type="submit" name="submit" value="Register" />
   <input type="reset" value="clear"/>
</form>
```



Exercise: connectdb.php

```
<?php
// Creates a PDO object called $db and establishes the MySQL
database connection
try{
 $db = new PDO("mysql:host=localhost;dbname=club", "root", "");
 $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $ex) {
  ?>
    <p>Sorry, a database error occurred.</p>
       <p> Error details: <em> <?= $ex->getMessage() ?> </em></p>
<?php
}
?>
```

Exercise: register.php -1

```
<?php
include("connectdb.php");
#Get form data and do form validation:
if (isset($_POST['username'])){
          $username = $_POST['username'];
     } else {
          echo "Pls input your user name";
          exit;
     }

if (!empty(trim($_POST['password']))){
          $password = password_hash($_POST['password']);
   }else {
          echo '<p><b>You forgot to enter your password!</b></p>';
          exit;
    }

$gender =$_POST['gender'];
$program=$_POST['programme'];
$reason=$_POST['Reason'];
……
```

Continue at next page

```
  try{
    # insert a member record based on the form data
    $sth=$db->prepare("INSERT INTO members
      VALUES('',:username,:password,:programme,:reason,:gender) ");
    $sth->bindParam(':username', $username, PDO::PARAM_STR, 10);
    $sth->bindParam(':password', $password, PDO::PARAM_STR, 64);
    $sth->bindParam(':programme', $program, PDO::PARAM_STR, 20);
    $sth->bindParam(':reason', $reason, PDO::PARAM_STR, 100);
    $sth->bindParam(':gender', $gender, PDO::PARAM_STR, 10);

    $sth->execute();

  } catch (PDOException $ex) {
  //this catches the exception when it is thrown
?>

    <p>Sorry, a database error occurred. Please try again.</p>
    <p>(Error details: <?= $ex->getMessage() ?>)</p>

<?php
  }
?>
```

Hash your passwords!

- Passwords should never be sent or stored in plain text!
- **Hashing** algorithms take an input and return a hashed value
  - `hash(string $algo, string $data[, bool $raw_output=false])`

```
hash('md5', $password);  #simple not very safe
#using md5 algorithm returning a 32-charater hex number
```

  - **`password_hash(string $password, int $algo [,array $options])`**

```
password_hash($password, PASSWORD_DEFAULT); #recommended
```

#The DEFAULT algorithm is BCRYPT which will produce a 60 character result.

- Verifies a password
  - **`password_verify ( string $password, string $hash )`**

```
password_verify($plainpass,$hashedpass);
```

 # Verifies that the given plain password matches the hashed password

PHP Password Hashing: https://www.php.net/manual/en/book.password.php

Outline

- Working with databases
- Cookies & Sessions
- Authentication using sessions
- OO PHP
  - Procedural *vs* Object-Oriented
  - Define Classes and Objects

HTTP is stateless!

- HTTP was originally designed to be stateless.
- Stateless protocols treat every interaction as a new request. No information is passed from one to another
- HTTP only allows a browser to request a single document from a web server
- The server forgets everything about from the client.
- **Cookies** and **Sessions:** two methods used to pass information between pages
- Other methods of passing information: hidden fields in form and query strings

Cookies

What are cookies?

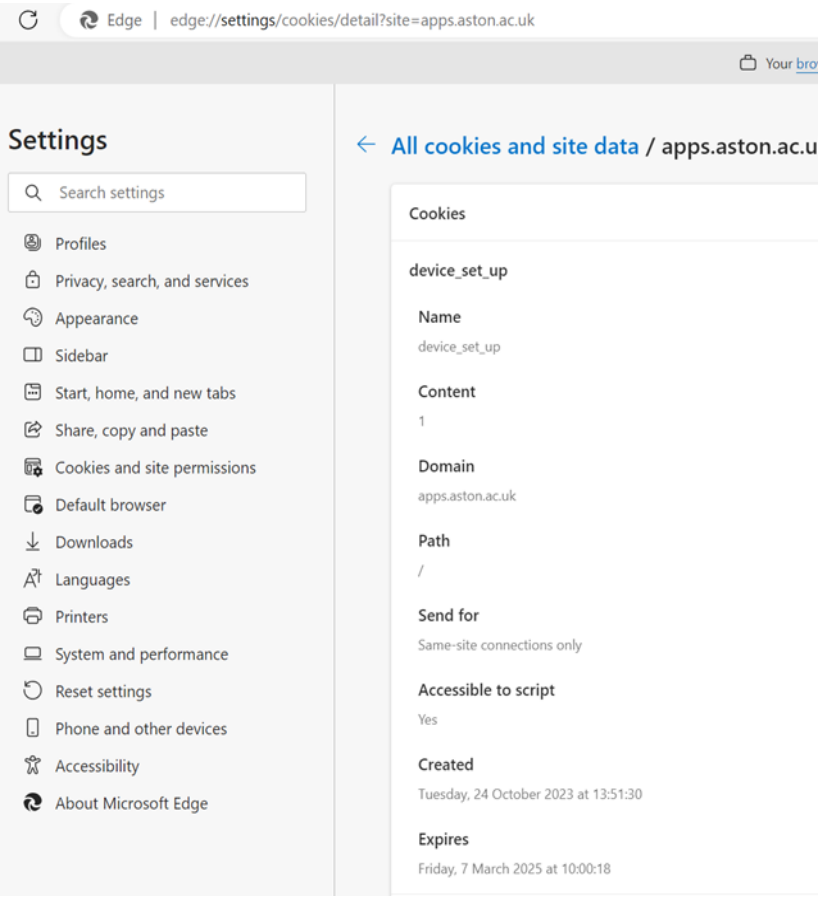
- Small pieces of information about a user that are stored **on the user's computer** in text files by a Web server

Why use cookies?

- A Web page script can read the previously stored browser cookie data
- Cookies can be used for authentication, user tracking and maintaining user preferences, shopping carts
  - Query strings can become too long
  - They are lost when someone closes a tab, can be difficult to get back

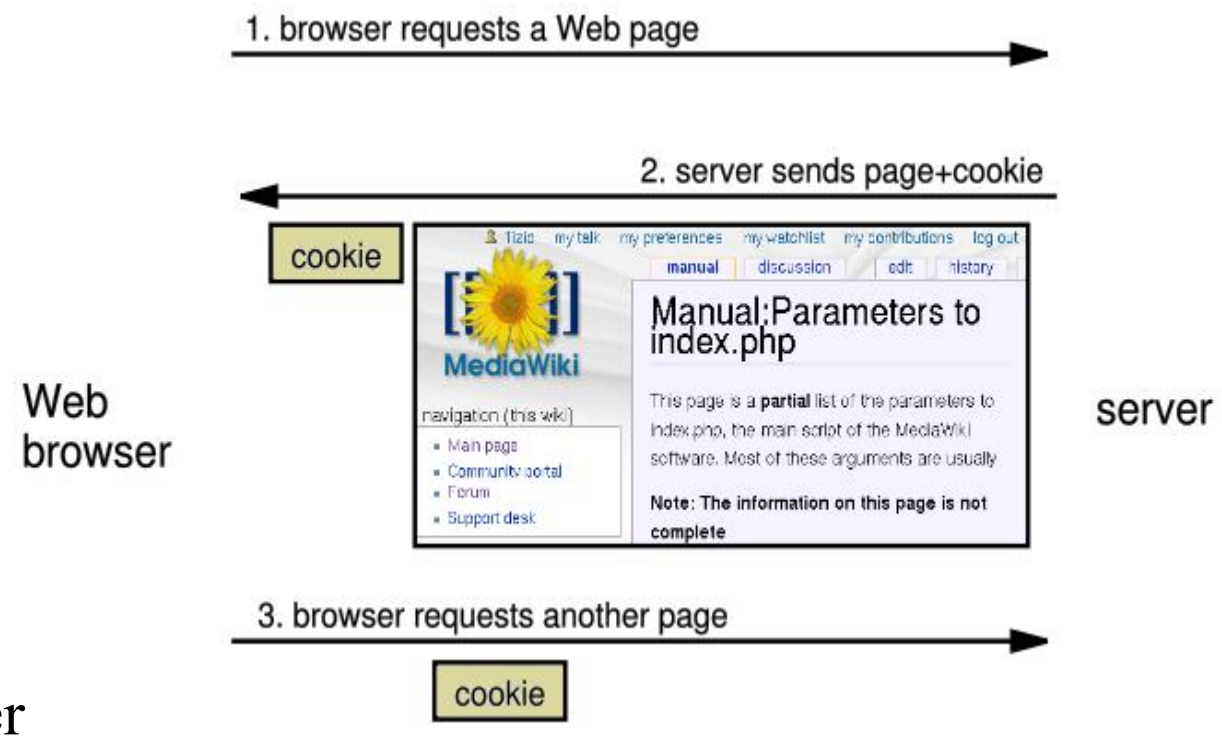
What is stored?

- A cookie's data consists of a single name/value pair, sent in the header of the client's HTTP request



How cookies are sent

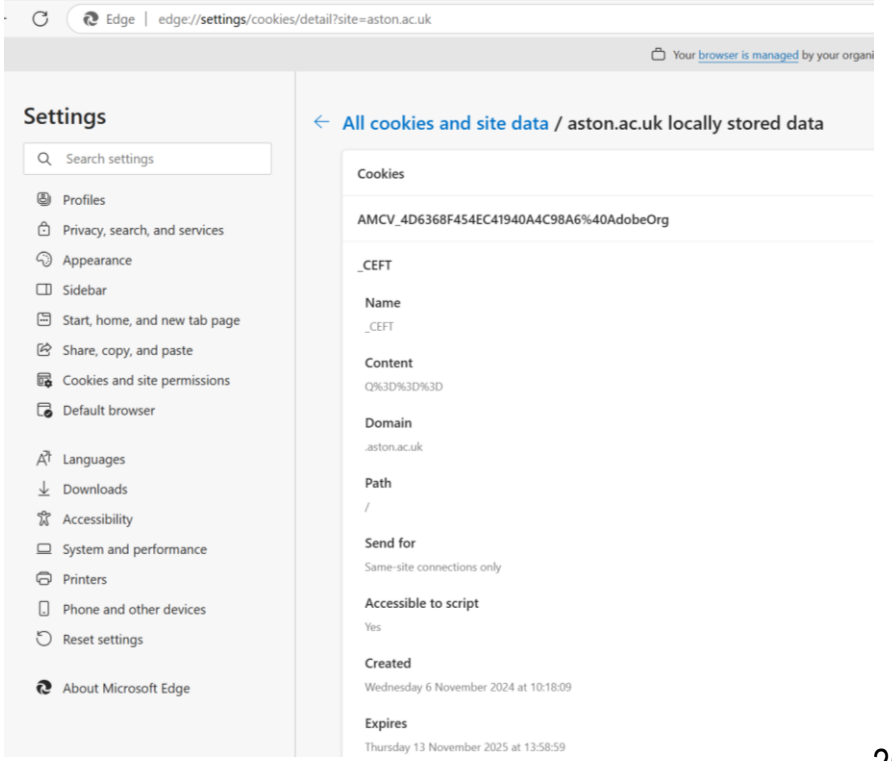
- When the browser requests a page, the server may send back a cookie(s) with it
- If your server has previously sent any cookies to the browser, the browser will send them back on subsequent requests



Setting Cookies using PHP

setcookie(name[,**value**,expires,path,domain])

- **expires** determines how long a cookie can remain on a client system before it is deleted.
  - If not included the cookie expires when the page is closed



- **path** argument determines the availability of a cookie to other Web pages on a server
- **domain** is for sharing cookies across servers in the same domain

setcookie() examples

```
setcookie("user", “Mike");
setcookie("pie", "apple");
```

#Cookies created without an expires argument are available for only the current browser session.

```
setcookie(“user", "Mike", time () + 3600);
```

#expires in one hour (60 sec * 60 mins) added to the current time()

```
setcookie(“user", "", time()-3600)
```

#delete this cookie

How long does a cookie exist?

- Session cookie: the default type; a temporary cookie that is stored only in the browser's memory
  - when the browser is closed, temporary cookies will be erased
  - cannot be used for tracking long-term information
  - safer, because no programs other than the browser can access them
- Persistent cookie: one that is stored in a file on the browser's computer
  - can track long-term information
  - potentially less secure, because users (or programs they run) can open cookie files, see/change the cookie values, etc.

Retrieving cookie information

- Cookies that are available to the current Web page are automatically assigned to the **$_COOKIE** super global array.
- Access each cookie by using the cookie name as a key in the associative $_COOKIE [ ] array
- `$variable= $_COOKIE["name"];`
- Newly created cookies are not available until after the current web page is reloaded.
- isset() finds out whether a cookie has been set

```
if(isset($_COOKIE["user"])) {
     $user= $_COOKIE["user"];
     echo "Welcome $user !\n";
} else
     echo "Welcome guest!\n";
print_r($_COOKIE);
```

Facts about Cookies

- Cookies are only data, not program code.
- Cookies cannot erase or read information from the user's computer.
- Cookies are usually anonymous (do not contain personal information).
- Cookies **CAN** be used to track your viewing habits on a particular site.
- Each individual server or domain can store around 20-50 cookies on a user's computer
- The largest cookie size is around 3-4K bytes.

Cookie Limitations

- Users can easily disable the cookies feature
- People move around
- Users may delete cookies
- PHP sets limit size on cookies
- You have to set it before you can send HTTP headers
- Less secure than session
- Session variables in PHP overcome some of the limitations of cookies.

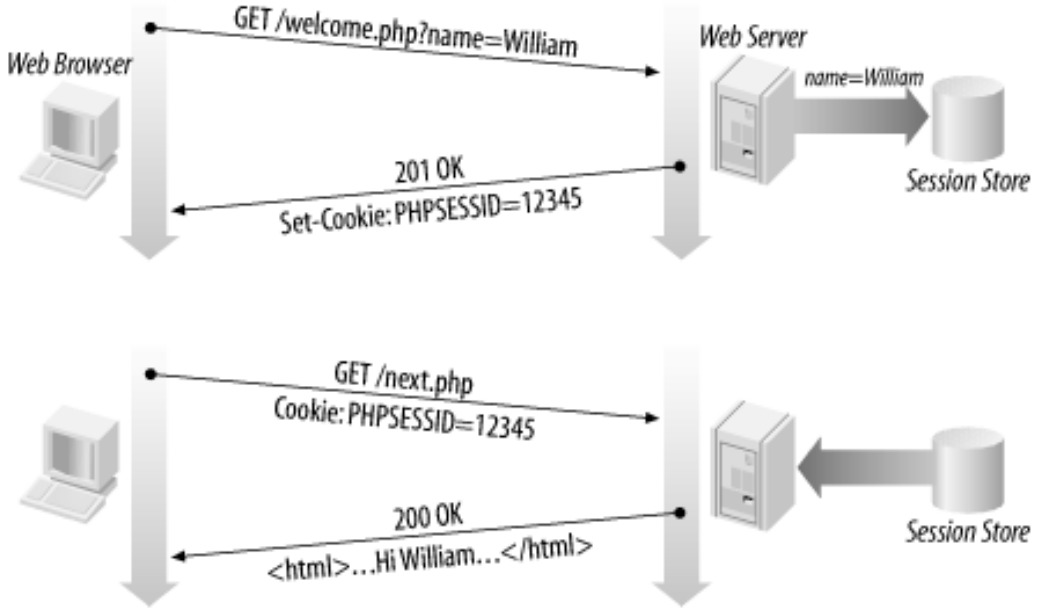
What is a session?

A session is a sequence of HTTP requests and responses between a specific web browser and a server from one client until the client logs out, or times out.

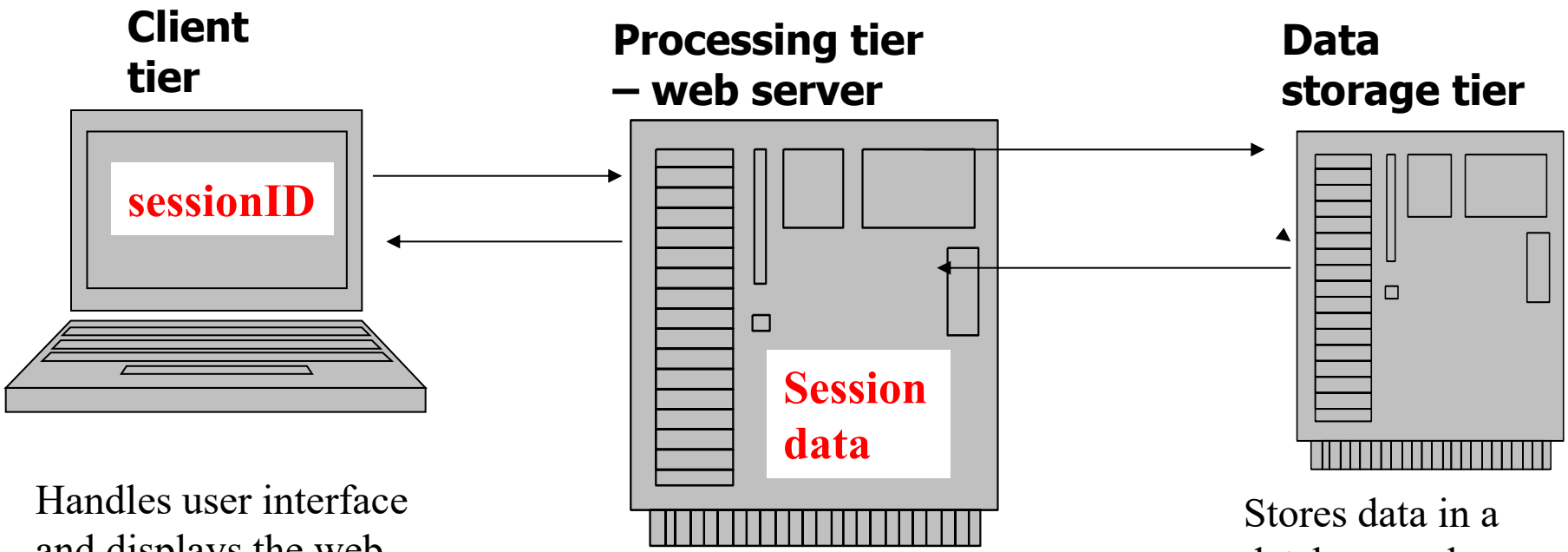
- **Sessions** are used to temporarily store data on the server
- Session allow you to keep track of your users even when clients disable cookies in their Web browsers
- **Session ID** serves as a reference to the session data
- Session ID is stored in the web browser via cookie (sessions are built on top of cookies)

How sessions are established

- Client's browser makes an initial request to the server
- Server notes client's IP address/browser, stores some local session data, and sends a session ID back to client
- Client sends that same session ID back to server on future requests
- Server uses session ID to retrieve the data for the client's session later



Client/Server Architecture (three tier)



Handles user interface and displays the web browser. Submits requests to the processing tier.

Handles interaction between the web browser client and the data storage tier

Stores data in a database and returns requests presented by the processing tier.

Setting Session Variables in PHP

- The script needs to start the session using `session_start()`

when you call session_start:

  - if the server hasn't seen this before, a new session is created
  - otherwise, existing session data is loaded into **$_SESSION**

- $_SESSION global array storing session data which can be retrieved on future pages based on the session ID.
- Once the session has started, values can be registered to the session, e.g.

```
session_start();
$_SESSION ["user"] = "Mike";
```

- If a client's Web browser is configured to accept cookies, the **session ID** is assigned to a temporary cookie named PHPSESSID

Accessing Session Variables

- Session variables are available after you've established them

```
echo "Hi $_SESSION ["user"] ";
```

- Use `isset()` to ensure that a session variable is set before you attempt to use it.

```
$_SESSION["name"] = value;  # store session data
$variable = $_SESSION["name"]; # read session

if (isset($_SESSION["name"])) { #check for session data
  $name=$_SESSION["name"];
   echo "welcome back, $name!\n";
} else {
   echo "Welcome your visit\n";
}
```

Modify / Delete Session Variables

- A session's data is temporary and does not require that you explicitly clean after yourself, but you may wish to delete some data for your various tasks.

```
// open the session to modify or remove it
session_start();

// to change a variable, just overwrite it
$_SESSION["user"]="Lucy";

// remove a single variable in the session
unset($_SESSION["user"]);

// Unset all of the session variables.
$_SESSION = array();

// this would destroy the session variables
session_destroy();
```

Complete list of session functions
http://uk.php.net/manual/en/ref.session.php

Using Sessions Without Cookies

- By default sessions rely on the use of a cookie `PHPSESSID` which contains session ID
- Every subsequent page that calls `session_start()` makes use of the cookie
- If users have cookies turned off, PHP will create a session for each page containing `session_start()` and none of the registered variables will be accessible

Using Sessions Without Cookies

- **Problem**: Users have cookies turned off!
- Solution: pass along the session ID as a query string from page to page
- PHP helps to automate this process with the built-in SID constant.
- If the browser supports cookies, this constant is empty; otherwise, SID contains a string similar to the following:
  - ***PHPSESSID=illijfvlsqnbtcjnr5ei10p8u7***

```
<a href="myscript.php?<?= SID ?> "> HOME PAGE </a>
```

Sessions vs. Cookies

- Data is retained on the server
- Data last until user closes the browser or times out
- Sessions are generally more secure
- Sessions allow for more data to be stored
- Sessions can be used without cookies.
- For most of our web applications, use sessions.
- Sensitive data, use sessions

- Data stored on client
- Cookies are easier to program
- Cookies require less of the server
- Cookies can be set to a long lifespan
- In general to store and retrieve just a couple of small pieces of information use cookies.
- If you want data work next day, use cookies

Outline

- Working with databases
- Cookies & Sessions
- Authentication using sessions
- OO PHP
  - Procedural *vs* Object-Oriented
  - Define Classes and Objects

Web Authentication

Authentication: tell a web application about who you are

- **Basic authentication (BA):** defined in HTPP and uses standard fields in the HTTP header, removing the need for handshakes.
- **Form-based authentication:** a programmatic method normally use the standard HTML form fields to pass the username and password values to the server and the server validates the credentials and then creates a "session" that is tied to a unique key that is passed between the client and server on http requests.
- **Token-based authentication for API:** each request to a server is accompanied by a signed token which the server verifies for authenticity and only then responds to the request.

A PHP authentication example

- Login Page using Form an Databases

Login

User Name:

Password:

Submit

```
<form action='login.php' mehtod ='post' >
     <label> Username:</label><br/>
     <input type="text" name="user" /> </br>
     <label> Password:</label><br/>
     <input type="password" name="passw" /> </br>
     <input type="submit" value="Login" />
     <input type="hidden" name="submitted" value="true"/>
</form>
```

Example: Login.php

```
if (isset($_POST ["submitted"])) {
   require_once ("connectdb.php");
//Query the matching username/password by prepare prevent SQL injection.
//code simplified here without considering the password hash
   $stat = $db->prepare('SELECT username, programme FROM members
     WHERE username = ? and password =?');
   $stat->execute(array($_POST['user'],$_POST['passw'] ));
   // fetch the result record to $row
   $row = $stat->fetch(); // simplified by assuming $query is valid
   session_start(); // Set the session data
   $_SESSION["user"] = $row['username'];

   header("Location: loggedin.php");//Redirect to loggedin.php
   exit(); // Quit the script.
}
$page_title = "Login";
?>
<form action="Login.php" method="post">
   //form elements here
</form>
```

Web Browser Redirection

- PHP `header()` can be used to redirect the Web browser from the current page to another

```
header ("Location: $url");
```

- Because redirection is the **last thing** to occur on the 'current page', header function is normally followed by exit () which stops execution of the current script.

- header () function must be called before anything is written to the page (including HTML). Call it before the <HTML> tag.

Example: loggedin.php

```
session_start(); // Start the session.

if (!isset($_SESSION["user"])) {
   // If no session value is present, redirect the user
   header("Location: sessionLogin.php");
   exit(); // Quit the script.
}
// Set the page title and include the HTML header.
$page_title = "Logged In! ";
include ("header.php");

// Print a customized message.
echo "<h1>Logged In!</h1>
<p>You are now logged in, {$_SESSION ["user"]}!</p>";
```

Outline

- Working with databases
- Cookies & Sessions
- Authentication using sessions
- OO PHP
  - Procedural *vs* Object-Oriented
  - Define Classes and Objects

PHP Programming So Far

- Write the initial HTML part
- Write some embedded PHP code blocks
  - Define functions
  - Include external files
  - Connects to the database
  - Queries/update/insert the databases
  - Retrieves the query results
  - Prints the results within some HTML (e.g. a table or a list)
- Completes the HTML page.

Procedural *vs* Object-Oriented

- Top-down design
- Function as programming unit
- Local data focused
- Limited code reuse
- Complex code

- Object focused design
- Classes as programming unit
- Protected data
- Code reuse
- Complex design

- The original PHP was written in the C programming language by Rasmus Lerdorf in 1994.
- From PHP5, OO features was fully implemented in 2004
- OOP offers encapsulation, inheritance, polymorphism and composition and can help in building extendable, reusable, maintainable larger applications.

Class Declaration Syntax

```
class ClassName {
        # fields - data inside each object
        public $vname1;  # public field, can be accessed everywhere
        private $vname2;  # private field, only accessed by the class

        # constructor - initializes each object's state
        public function __construct(parameters){
                statements;
        }

        # method - behavior of each object
        public function fname(parameters) {
                statements;
        }
}
```

Constructing and using objects

construct an object

```
$name = new ClassName(parameters);
```

access an object's field

```
$name->fieldName
```

call an object's method

```
$name->methodName(parameters);
```

Defining a PHP Class: Example

```
<?php
    class Bunny{  #define a class
      //variables - fields here
      private $name, $age;
      //methods
            ……
    }
?>
```

- This creates two variables, one name and another age
- Remember – we don't need to define variable types in PHP

Constructors

- The constructor is called when the object is created. This "sets up" the object for use.

```
public function __construct($x, int $y) {
    #Note: Function parameters can have type declaration and default value
    $this->name = $x;  #no need the $ for the variables
    $this->age = $y;
}
```

- $this is a reference to the calling object
- If you don't use $this->variable_name then a local variable is created inside the function; the values will then stop existing after the end }

Getter and Setter Methods

Getter method: used to retrieve (read) the value of a private attribute.

Setter method: used to update (modify) the value of a private attribute.

Two approaches:

- **First approach**: similar to other languages

```
//getters and setters
public function getName(){
 return $this->name;
}
public function setName($name){
 $this->name = $name;
}
```

Note – every function returns something, even if this hasn't been defined!

Writing boiler plate code is boring and prone to mistakes!

Getter and Setter Methods

- **Second Approach**: to pass variable or class names as parameters!

```
public function __get($var){
 return $this->$var;
}

public function __set($var, $value){
 $this->$var = $value;
}
```

- The methods beginning with two underscores (__) are called **Magic Methods** in PHP.

Using the Object

```
//create the object (uses the __construct())
$bunny = new Bunny("Angus", 1);

//get the name (uses the __get())
echo $bunny->name . "<br/>";

//set the name (uses the __set())
$bunny->name = "Pancakes";

//get the name again
echo $bunny->name . "<br/>";
```

Output:

"Angus
Pancakes"

- `new` will call the `__construct()` method.
- `$name` is a private field, need the `__get()` or `__set()` method to access it outside class.

  e.g. `echo $bunny->name` will call the `__get()` method

Class Definition and Usage Example

```
class Bunny{
  private $name;
  private $age;

  public function __construct($x, $y) {
        $this->name = $x;
        $this->age = $y;
   }
  public function __get($var){
        return $this->$var;
  }

   public function __set($var, $value){
        $this->$var = $value;
   }
```

```
$bunny = new Bunny("Angus", 1);
echo $bunny->name . "<br/>";
$bunny->name = "Pancakes";
echo $bunny->name . "<br/>";
```

Output:

Angus
Pancakes

The __ notation: Magic methods

- PHP reserves all function names starting with __ as **magical**.

e.g. __construct(), __destruct(), __get(),__set(), __toString(), etc.

- The two underscores before a method name means you do not directly call the method.
- __toString() method: decide how to react when one object is treated like a string.

For the bunny class:

```
public function __toString() {
    return $this->name.' is '.$this->age. ' years old <br>';
  }

$bunny1 = new Bunny('Star', '2');
echo $bunny1;
```

Star is 2 years old

Debugging Methods

- `print_r($bunny)`
  - prints out human readable object information

```
Bunny Object ( [name:Bunny:private] => Pancake
[age:Bunny:private] => 1 )
```

- `var_dump($bunny)`
  - prints out detailed object information

```
object(Bunny)[1]
  private 'name' => string 'Pancakes' (length=8)
  public 'age' => string '1' (length=1)
```

- Use this to provide useful information about the state of your system

Class Inheritance

- Inheritance allows objects to exist in a hierarchical structure
- You can have some methods and variables in common, but also deal with other methods in different ways
- Example: Animals all have a name and an age, but they do different things and make different noises.

Inheritance in PHP - Parent

```
abstract class Animal {
        private $name;
        private $age;
        //Constructor
        public function __construct($x, $y) {
                $this->name = $x;
                $this->age = $y;
        }
        //getters and setter etc.
        public function __get($var){
                return $this->$var;
        }
        public function __set($var, $value){
                $this->$var = $value;
        }
        public abstract function getNoise();
}
```

Inheritance in PHP - Children

```
class Bunny extends Animal {

 public function getNoise(){
  return "Snuffle";
 }
}
```

```
$bunny = new Bunny("Angus", 1);
echo $bunny->name." goes ".$bunny->getNoise()."<br>";
```

Angus goes Snuffle.

Inheritance in PHP - Children

```
class Dog extends Animal {
  public $role;
  public function __construct($name, $age, $role) {
        parent::__construct($name, $age);
        $this->role=$role;
  }
  public function getNoise(){
     return "Bark";
  }
}
```

```
$dog1 = new Dog("Beast", 3, "hunting");
$dog2 = new Dog("Honey", 8, "companionship");

echo $dog1->name." is ".$dog1->age. "and does ".$dog1->role.". </br>";
echo $dog2->name . " goes " . $dog->getNoise();
```

Beast is 3 and does hunting.
Honey goes Bark.

Static fields and constants

- Static fields in a class

```
static $name = "admin"; # declaring a static field
#Static variable is accessible without making an instance of a class.
```

- Class constant

```
const NAME= "public"; # declaring a constant field
#class constants are allocated once per class, and not for each class instance.
```

- Using a static field

```
ClassName::$name; #outside class
Self::$name; #within class
```

- Using a constant

```
ClassName::NAME; #outside class
Self::NAME; #within class
```

Static Methods

Static methods aren't associated with an instance; they belong to the class.

```
# declaring a static method
public static function name(parameters) {
     statements;
}
# calling a static method (outside class)
ClassName::methodName(parameters);
# calling a static method (within class)
self::methodName(parameters);
```

Summary

- Working with databases
  - PDO and PDOstatement
  - SQL injection
- Hash password
- Cookies & Sessions
  - Concepts and difference
  - how to use them and use
- Authentication using sessions
- OO PHP
  - Procedural vs Object-Oriented
  - Define Classes and Objects